Laboratory 1

Variant 2, Group 32

By Adam Shahumi and Santiago Puig-Terrero López

1. INTRODUCTION

The aim and objective of this lab exercise is to create an intelligent agent that can play the game of Connect 4 with a human player by implementing the minimax search algorithm with alpha-beta pruning.

Connect 4 is a deterministic two-player zero-sum game with perfect information, making it a classic example of an adversarial search problem in the field of Artificial Intelligence. The game is played on a 6x7 grid, where players take turns to drop their pieces vertically into an empty column, with the piece being dropped to the lowest available cell. The first player to get four pieces consecutively horizontally, vertically, or diagonally wins the game, while a drawn game occurs if all the cells are filled and no player has got four pieces consecutively.

In this version, a depth-limited minimax algorithm with alpha-beta pruning is used to limit the number of nodes to be explored and thereby increase the depth of look-ahead in a reasonable time. At non-terminal nodes, a heuristic evaluation function is used to estimate how good a position is for one of the players. This function is based on examining small “windows” of four consecutive cells on the board and counting how many have three-in-a-row with a free space or potential four-in-a-row for both players. The aim of this report is to describe the problem and algorithm used, show some examples of test cases for the agent, and discuss what has been learned from this assignment.

2. IMPLEMENTATION

The solution is built using a Python class named ConnectFour to encapsulate the board state and the AI logic, alongside a main game loop for user interaction.

Board Representation and Mechanics

The board is represented as a 6x7 2D list. We implemented helper functions to govern the rules of Connect 4:

- is_valid_location: checks if a column has space,
- get_next_open_row: simulates gravity by finding the lowest empty row in a column,
- winning_move: checks all horizontal, vertical, and diagonal lines for 4 consecutive identical pieces.

```python
def winning_move(self, board, piece):
    """Check the board for a 4-in-a-row horizontal, vertical, or diagonal."""
    # Horizontal
    for c in range(COLS - 3):
        for r in range(ROWS):
            if board[r][c] == piece and board[r][c+1] == piece and board[r][c+2] == piece and board[r][c+3] == piece:
                return True
    # Vertical
    for c in range(COLS):
        for r in range(ROWS - 3):
            if board[r][c] == piece and board[r+1][c] == piece and board[r+2][c] == piece and board[r+3][c] == piece:
                return True
    # Positive Slope Diagonal
    for c in range(COLS - 3):
        for r in range(ROWS - 3):
            if board[r][c] == piece and board[r+1][c+1] == piece and board[r+2][c+2] == piece and board[r+3][c+3] == piece:
                return True
    # Negative SlopeDiagonal
    for c in range(COLS - 3):
        for r in range(3, ROWS):
            if board[r][c] == piece and board[r-1][c+1] == piece and board[r-2][c+2] == piece and board[r-3][c+3] == piece:
                return True
    return False
```

Evaluation Function

To evaluate non-terminal board states, we split the board into "windows" of 4 cells (horizontal, vertical, and both diagonals). The evaluate_window function assigns scores to these arrays:

- 4 pieces of the same kind: +100 points
- 3 pieces + 1 empty space: +5 points
- 2 pieces + 2 empty spaces: +2 points
- Opponent has 3 pieces + 1 empty space: -4 points (to heavily prioritize blocking) Additionally, evaluate_position grants a +3 point bonus for every piece placed in the center column, encouraging the AI to play centrally where the most winning combinations intersect.

NOTE: Please note that these values are thought out/planned, but also somewhat arbitrary and could be changed/tweaked depending on the idea for this algorithm, however we decided that this was a good baseline to work with.

```python
def evaluate_window(self, window, piece):
    """
    Evaluation of given window. Helper function to evaluate the separate parts of the board called windows
    """
    score = 0
    opp_piece = PLAYER_O if piece == PLAYER_X else PLAYER_X

    if window.count(piece) == 4:
        score += 100
    elif window.count(piece) == 3 and window.count(EMPTY) == 1:
        score += 5
    elif window.count(piece) == 2 and window.count(EMPTY) == 2:
        score += 2

    # Block the opponent
    if window.count(opp_piece) == 3 and window.count(EMPTY) == 1:
        score -= 4

    return score
```

Minimax Algorithm with Alpha-Beta Pruning

The core intelligence is driven by the minimax recursive function. It simulates all possible valid moves up to a defined depth (set to 5 in our main loop). It alternates between the maximizing player (the AI attempting to get the highest score) and the minimizing player (assuming the human will make the best move to lower the AI's score). Alpha-beta pruning is used to stop evaluating branches of the game tree that are mathematically proven to be worse than a previously examined branch, significantly increasing the speed of the AI.

Since the method is quite complex and long, please refer to the source code file for more details on how it was coded exactly.

Game Loop and Testing

The main() function handles the text-based console interface, capturing human input, handling invalid typing via try-except blocks, and updating the board visual. An automated testing function was also implemented to instantly load predetermined array states and verify the AI's correct responses.

3. DISCUSSION

In order to test the behavior of the Connect 4 agent, several test cases have been defined, which cover different types of situations like initial moves, winning moves, defensive moves, etc. In each of these test cases, we discuss what an "ideal" move is, and how the minimax agent behaves based on the chosen evaluation function and the chosen depth of the search.
NOTE: The results generated by this automated test and the printed messages will be shown at the end.

3.1. Empty board (opening move)

In the first test case, the game begins with the regular empty board of size 6x7, and the AI is prompted to make the first move. There are no pieces to attack or defend, and no immediate win to be made, only long-term potential. One good approach to playing Connect Four is to aim to place pieces in the middle columns, since those columns are part of more possible winning lines than the outside columns. This means that, during the minimax exploration of possible first moves, the central columns will have a very slightly better heuristic value than the other columns, and the AI will likely choose a central column for its first move.

3.2. Immediate win for the AI

In the second test case, we create a position where the AI already has three of its pieces in a row with one empty cell that will give the AI four in a row if filled. It is the AI's turn to move. Any rational player in such a position should select the move that will give them the win. With the current implementation of the minimax algorithm with alpha-beta pruning, the check for terminal states in the minimax algorithm will recognize that playing in the column where the empty cell is will give the AI the win, which is assigned a very high positive value in the evaluation function. The AI will indeed select the immediate winning move.

3.3. Immediate threat from the human

The third case focuses on defensive behaviour. We design this situation such that the human player has three in a row with an open cell which would give the human four in a row in the next turn if not blocked. Now it is the AI's turn. If the AI does not put its piece in the only column which blocks the human's four in a row, the human would win in the next turn. When the minimax algorithm searches the possible answers of the opponent in the next level in the minimax tree, the minimax algorithm evaluates the position which does not block the human player's four in a row with a very large negative score from the point of view of the AI because this position would allow the human player to win in the next turn. The position which blocks the human player's four in a row would not allow the human player to win in the next turn. Therefore, the minimax value of the position which blocks the human player's four in a row would be higher than the minimax value of the position which does not block the human player's four in a row.

3.4. Competing threats in mid-game

In a more complex mid-game test case, both players have several pieces placed, and the board contains potential threats for each side. For example, the AI might have a two-in-a-row that can be extended, while the human has a configuration that can lead to a three-in-a-row on the next move. No player can win immediately in a single move, but there are moves that improve their positions substantially. In such situations, the correct decision depends on the depth to which minimax searches and on how the evaluation function balances offensive and defensive patterns. With a reasonable depth, the AI can look ahead a few plies and compare sequences where it plays aggressively versus sequences where it first neutralizes the opponent's threats. If the heuristic gives a sufficiently high penalty to strong threats of the opponent (e.g. three-in-a-row with an empty cell) and high rewards to building its own such patterns, the agent often chooses a move that both prevents dangerous lines for the opponent and creates or maintains its own potential four-in-a-row. Analysing these positions helped to tune the relative weights used in the evaluation function so that blocking and attacking are balanced appropriately.

3.5. Draw case

The final test case considers a near-endgame position where the board is almost full and only one more move remains possible. In this position, there is not a realistic way for the AI nor the human to force a win. When the AI makes the next move, it only leads to a draw as the outcome. Therefore, the purpose of this case is to test whether the system correctly handles a draw situation.

```
1   ========================================
2         RUNNING AUTOMATED AI TESTS
3   ========================================
4
5   ------------------------------------------------
6   Running Test 1: AI must block an immediate horizontal win...
7
8   Board state BEFORE AI move:
9
10
11  | | | | | | | |
12  | | | | | | | |
13  | | | | | | | |
14  | | | | | | | |
15  |O|O| | | | | |
16  |X|X|X| | | | |
17  ---------------
18   0 1 2 3 4 5 6
19
20
21  AI (O) is thinking...
22
23  Board state AFTER AI drops piece in column 3:
24
25
26  | | | | | | | |
27  | | | | | | | |
28  | | | | | | | |
29  | | | | | | | |
30  |O|O| | | | | |
31  |X|X|X|O| | | |
32  ---------------
33   0 1 2 3 4 5 6
34
35
36  ✅ PASS: AI chose column 3 as expected.
```

```
38  ------------------------------------------------
39  Running Test 2: AI must block an immediate vertical win...
40
41  Board state BEFORE AI move:
42
43
44  | | | | | | | |
45  | | | | | | | |
46  | | | | | | | |
47  | | | | |X| | |
48  | | | | |X| | |
49  | | |O|O|X| | |
50  ---------------
51   0 1 2 3 4 5 6
52
53
54  AI (O) is thinking...
55
56  Board state AFTER AI drops piece in column 4:
57
58
59  | | | | | | | |
60  | | | | | | | |
61  | | | | |O| | |
62  | | | | |X| | |
63  | | | | |X| | |
64  | | |O|O|X| | |
65  ---------------
66   0 1 2 3 4 5 6
67
68
69  ✅ PASS: AI chose column 4 as expected.
70
```

```
--------------------------------------------------
Running Test 3: AI takes an immediate win instead of blocking...

Board state BEFORE AI move:


| | | | | | | |
| | | | | | | |
| | | | | | | |
|X|O| | | | | |
|X|O| | | | | |
|X|O| | | | |X|
---------------
 0 1 2 3 4 5 6


AI (O) is thinking...

Board state AFTER AI drops piece in column 1:


| | | | | | | |
| | | | | | | |
| |O| | | | | |
|X|O| | | | | |
|X|O| | | | | |
|X|O| | | | |X|
---------------
 0 1 2 3 4 5 6


✅ PASS: AI chose column 1 as expected.
```

```
--------------------------------------------------
Running Test 4: AI as Player 1 (X) builds a vertical win...

Board state BEFORE AI move:


| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | |X| | | |
| | |O|X| | | |
| | |O|X|O| | |
---------------
 0 1 2 3 4 5 6


AI (X) is thinking...

Board state AFTER AI drops piece in column 3:


| | | | | | | |
| | | | | | | |
| | | |X| | | |
| | | |X| | | |
| | |O|X| | | |
| | |O|X|O| | |
---------------
 0 1 2 3 4 5 6


✅ PASS: AI chose column 3 as expected.
```

```
--------------------------------------------------
Running Test 5: Draw case (Board almost full, 2 moves left)...

Board state BEFORE AI move:


|O|X|O|X|O|X| |
|O|X|O|X|O|X| |
|O|X|O|X|O|X|O|
|X|O|X|O|X|O|X|
|X|O|X|O|X|O|O|
|X|O|X|O|X|O|X|
---------------
 0 1 2 3 4 5 6


AI (X) is thinking...

Board state AFTER AI drops piece in column 6:


|O|X|O|X|O|X| |
|O|X|O|X|O|X|X|
|O|X|O|X|O|X|O|
|X|O|X|O|X|O|X|
|X|O|X|O|X|O|O|
|X|O|X|O|X|O|X|
---------------
 0 1 2 3 4 5 6


✅ PASS: AI chose column 6 as expected.

========================================
TEST RESULTS: 5 / 5 tests passed.
========================================
```

3.6 Strengths and Weaknesses of the Evaluation Strategy

- **Strengths:** The window-based evaluation is computationally lightweight, allowing the AI to search deeper within the same time limit. By assigning a negative penalty (-4) to opponent threats, the AI balances defensive blocking with offensive play. The center-column multiplier effectively mimics human opening strategies.

- **Weaknesses:** The evaluation function is highly localized; it only looks at 4 cells at a time. It cannot recognize complex "traps" (like a 7-move forced win) unless the search depth reaches the end of the trap. It also treats all 3-in-a-rows similarly, without prioritizing lower rows that will be filled sooner than higher rows.

4. CONCLUSION

- **What we learned**

Throughout the process of this lab, we learned how to apply the minimax algorithm with alpha-beta pruning to a real two-player game, Connect 4. We also learned the importance of the depth of the search and the alpha-beta pruning in the application of the minimax algorithm in a real-time game. Additionally, we learned the importance and application of the heuristic evaluation function. By counting the number of local patterns (two-, three-, and four-in-a-row for both players), the agent was able to estimate the value of a non-terminal state and thus make decisions that seemed to be "strategic" to a human player. Finally, the design of the test cases helped us understand the theory in practice in common game situations such as immediate win, block, and draw.

- **What was the challenge**

The main challenge was to create an evaluation function which strikes the right balance between offense and defense. Another challenge was to determine the depth of the search tree. The depth must be sufficiently large to allow the agent to see the necessary tactics but not so large that the search becomes unbearably slow. The branching factor in Connect 4 can be quite large in the early and mid-game. Another challenge was to create realistic test boards which adhere to the rules of the game and verify our winner detection logic.

- **A little extra**

A few real games were performed with the AI for testing purposes and also a little fun. While the testing was not done by “professional” connect 4 players, it was still done by humans who are somewhat knowledgeable about these sort of planning and logic games. As the creators of the program, we are happy to inform that despite genuinely trying to win, it was never possible

to beat the AI in the game! Below is an example of one such game and it was observed that the AI knows quite well how to force players into situations that no matter where the placement is done, it is impossible to escape a loss, hence we are satisfied with just adding a 5 step “future sight” for the algorithm and being more than enough to beat anyone who isn’t very serious about winning!

```
Computer is thinking (X)...
Computer drops a piece in column 2



| | | | | | | |
| | | |X|X| | |
| | | |X|O|X| |
| |O| |O|X|O| |
|O|X|X|X|X|O| |
|O|O|O|X|X|O| |
--------------
 0 1 2 3 4 5 6



Game Over. The computer wins!
```